تقرير العمل التطبيقي الأول (TP1):
نواة نظام إدارة حسابات الطلبة UniAccounts باستخدام أنماط الإنشاء (Creational Patterns)

من إعداد الطالبة: سليمة جيدور
الفوج: الفوج الأول (Group 01)
السنة الجامعية: 2025 /2026

1. تقديم المشروع والهدف من العمل

نماذج التصميم (Design Patterns)، يهدف هذا التقرير إلى تقديم وتوثيق نواة برنامج UniAccounts المخصص لإدارة حسابات الطلبة في الجامعة.

يواجه بناء الأنظمة البرمجية الواقعية عدة تحديات تتعلق بكيفية إنشاء الكائنات وإدارتها في الذاكرة دون الوقوع في مشاكل التعقيد أو ضياع البيانات. ركّز هذا العمل الأول على استخدام ثلاثة أنماط تصميم إنشائية (Creational Patterns) حقيقية وهي:

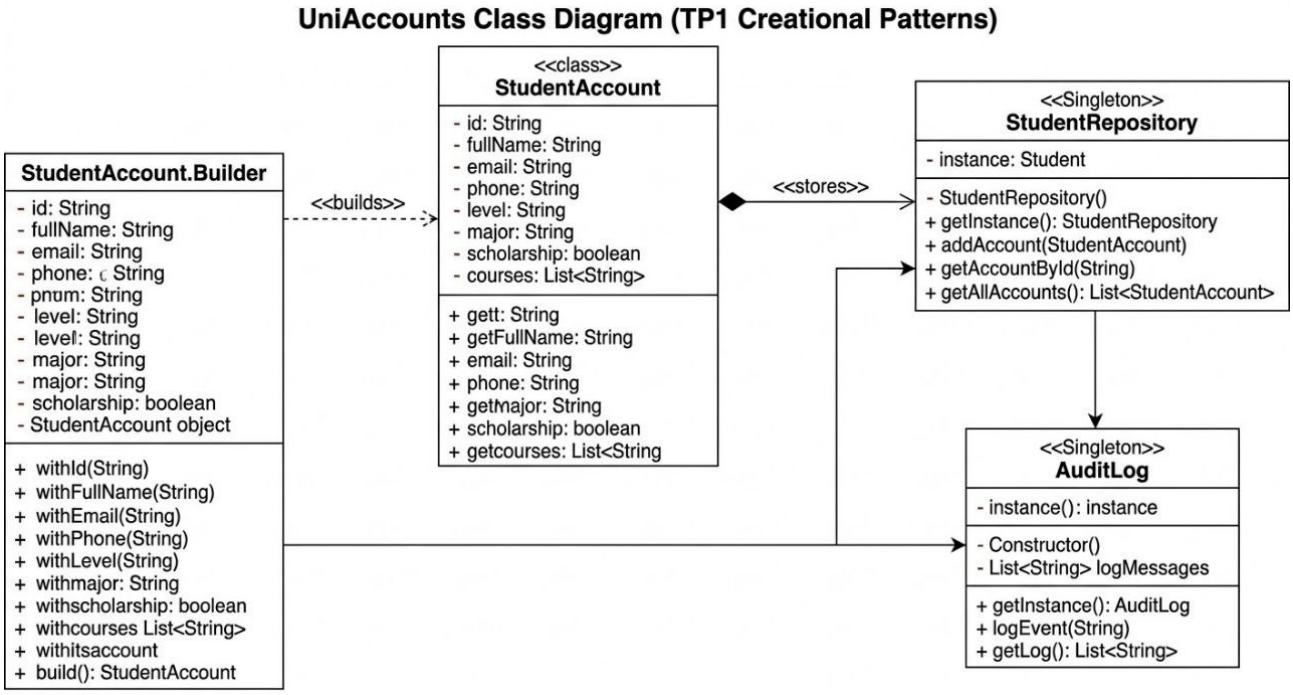
نمط البنّاء (Builder Pattern): لبناء كائن الطالب خطوة بخطوة وتفادي مشاكل المنشئات ذات الوسائط الكثيرة.

نمط النواة الوحيدة (Singleton Pattern): لضمان وجود مستودع مركزي واحد للحسابات وسجل تدقيق واحد للعمليات.

نمط النمط الأولي (Prototype Pattern): لإنشاء الحسابات الجديدة عبر نسخ قالب جاهز مخصص لكل دفعة أو تخصص.

2. مخطط الأصناف (UML Class Diagram)

يمثل الشكل التالي مخطط الأصناف النهائي للنواة، حيث يوضح هيكل الأصناف الأربعة والعلاقات البرمجية القائمة بينها وفق أنماط التصميم المطبقة:



شرح مكونات المخطط والعلاقات:

StudentAccount.Builder و StudentAccount: يمثل الصنف الداخلي Builder علاقة بناء وتكوين (<< builds) مع الصنف الرئيسي StudentAccount، كما يمتلك الصنف الرئيسي دالة النسخ cloneAccount للتعبير عن نمط Prototype.

StudentRepository: يمتلك علاقة احتواء وتخزين مع صنف StudentAccount (1 stores *) لترسخ إدارة قائمة الحسابات، ويعتمد على منشئ خاص ومترجعة ساكنة getInstance لتحقيق نمط Singleton.

AuditLog: صنف وحدوي بنمط Singleton يرتبط بعلاقة استخدام (uses) مع StudentRepository لتسجيل كافة عمليات الإضافة فور حدوثها.

3. دراسة وتبرير أنماط التصميم المطبقة

3.1. نمط البنّاء (Builder Pattern)

أين تم استعمله؟

تم استعماله داخل الصنف StudentAccount عبر إنشاء صنف داخلي ساكن باسم StudentAccount.Builder.

ما المشكلة التي حلها؟

يحتوي صنف الطالب على 8 حقول تتنوع بين إجبارية (مثل: id, fullName, email) واختيارية (مثل: phone, level, major, scholarship, courses). إن استعمال منشئ (Constructor) واحد بثمانية وسائط يجعل الشيفرة البرمجية غير مقروءة ومعرضة لأخطاء الترتيب عند التمرير.

ماذا كان سيحدث لو لم نستخدمه؟

لكُنّا اضطررنا لكتابة عدة منشئات متداخلة (Constructor Chaining) أو الاعتماد على دوال التعيين (setters) بعد إنشاء الكائن، مما يفقد الكائن حالة الاتساق الذاتي (Consistency) ويمكّن العميل من إنشاء حسابات ناقصة للبيانات الإجبارية.

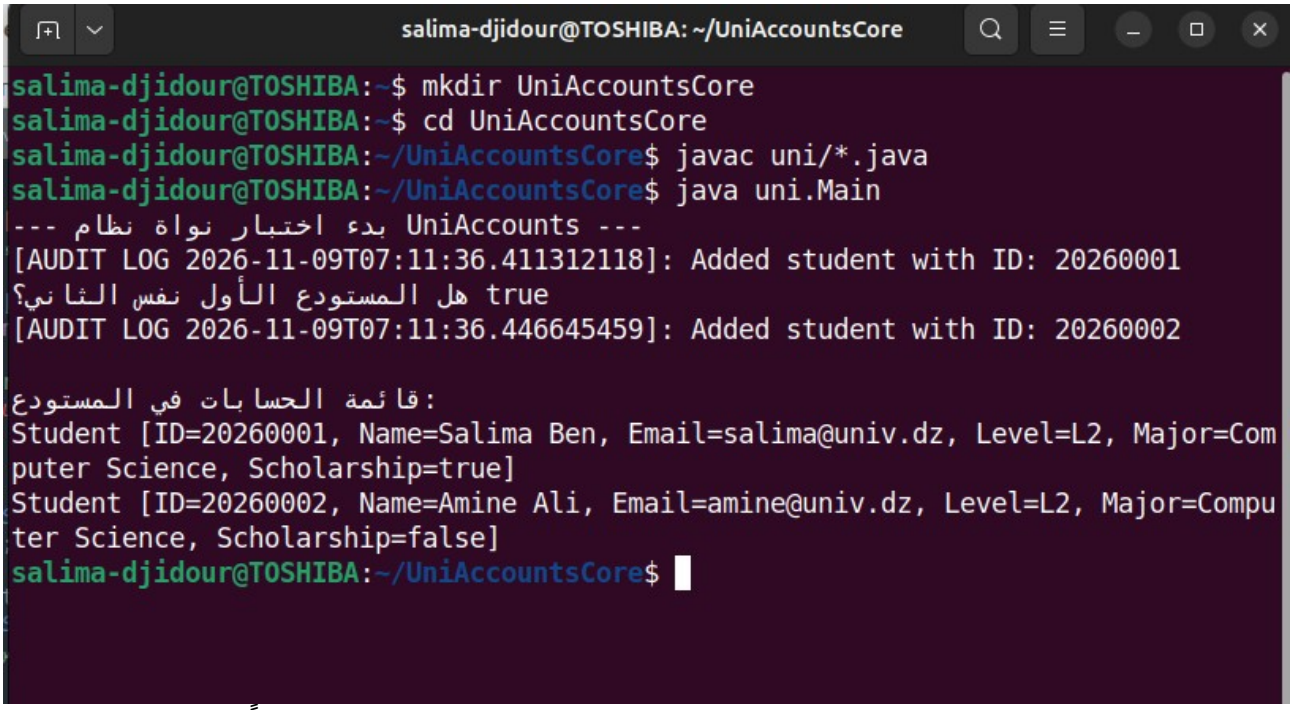
3.2. نمط النواة الوحيدة (Singleton Pattern)

أين تم استعمله؟

تم استعماله في صنفين مختلفين هما: مستودع الحسابات StudentRepository وسجل التدقيق AuditLog.

ما المشكلة التي حلها؟
ضمان وجود نسخة واحدة فقط في الذاكرة من مستودع الحسابات وسجل التدقيق، ومنع أي جزء من البرنامج من إنشاء نسخة جديدة تلغي أو تعزل البيانات الحالية.
ماذا كان سيحدث لو لم نستخدمه؟
لو أنشأ كل جزء أو صنف في البرنامج مستودعه الخاص أو سجل تدقيق منفصل بـ new، لضاعت بيانات الحسابات بين النسخ المختلفة ولأصبحت سجلات النظام تائهة وغير مركزية.
3.3. نمط النمط الأولي (Prototype Pattern)
أين تم استعمله؟
تم استعمله من خلال إضافة الدالة( ) cloneAccount داخل الصنف StudentAccount.
ما المشكلة التي حلها؟
في بداية كل سنة جامعية يلتحق مئات الطلبة بنفس التخصص والمستوى ولهم نفس المقررات والمنح. بدل بناء كل حساب طالب من الصفر وتعيين الخصائص المشتركة يدوياً، يسمح هذا النمط بإحضار حساب قالب (Template) ثم نسخه وتغيير هوية الطالب فقط (id, fullName, email).
ماذا كان سيحدث لو لم نستخدمه؟
لكان أداء النظام أبطأ وأكثر استهلاكاً للأوامر، حيث يتوجب على العميل إعادة تجميع البيانات وإدخال المقرر والتخصص والمستوى لكل طالب جديد على حدة.

4. نتائج تشغيل النواة (Execution Screenshots)
تمت ترجمة المشروع وتكشيف حزمة uni باستخدام مفسر الجافا
تحليل مخرجات الشاشة:



تفعيل الـ AuditLog: يظهر في السطر الأول تدوين عملية إضافة الطالب الأول تلقائياً بحساب توقيت الجهاز الدقيق [AUDIT LOG ...]: Added student with ID: 20260001.
إثبات نمط Singleton: عند مقارنة مرجع المستودع الأول بالمستودع الثاني عبر الشرط (repo1 == repo2) طبع البرنامج النتيجة true مما يثبت أن النسخة واحدة فقط في الذاكرة.
تفعيل الـ Prototype والـ Builder: تم إنشاء الطالب الأول عن طريق Builder والطالب الثاني عن طريق النسخ بـ cloneAccount واحتفظ بالخصائص الأكاديمية الموحدة (L2, Computer Science).